人工介入(Human-in-the-Loop)

- 来源: https://class.imooc.com/lesson/5285#mid=105038
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第9周 LangGraph：工作流编排
- 模块: 文章写作Agent
- 课节: 图文 1-3 人工介入(Human-in-the-Loop)

1-3 人工介入(Human-in-the-Loop)

人工介入(Human-in-the-Loop)

> [!TIP] 阅读指南 想象你在构建一个AI代码生成助手。测试阶段表现很好，但正式使用后发现:AI生成的代码虽然能运行，但有时会违反团队的代码规范，甚至引入潜在的安全漏洞。 你立刻意识到:AI虽然高效，但生成的代码仍需要人类审核。完全自动化可能带来不可控的风险。 解决方案是什么?加入人工介入(Human-in-the-Loop)——让AI生成代码后暂停，等待工程师审核通过后再继续执行。 这就是本节要解决的问题:如何在自动化工作流中优雅地加入人工控制点，实现人机协作。

核心概念

人工介入的关键机制

LangGraph通过编译时设置暂停点实现人工介入:

编译时设置暂停点

```
from langgraph.checkpoint.memory import MemorySaver

# 编译时指定暂停点
checkpointer = MemorySaver()  # 必需:保存状态快照
app = workflow.compile(
    interrupt_before=["review"],  # 在review节点前暂停
    checkpointer=checkpointer
)
```

两个关键要素:

两个关键要素

- interrupt_before: 指定在哪个节点前暂停工作流
- checkpointer: 保存工作流状态，支持暂停后恢复

执行模式

人工介入采用两次调用模式:

两次调用

```
config = {"configurable": {"thread_id": "session_001"}}

# 第一次调用:执行到暂停点
result = app.invoke({"requirement": "..."}, config)

# 工程师审核并更新决策
app.update_state(config, {"human_decision": "approve"})

# 第二次调用:从暂停点继续
result = app.invoke(None, config)  # None表示从检查点恢复
```

关键点:

关键点

- thread_id: 唯一标识一次工作流会话
- update_state(): 更新状态中的人工决策

```
update_state()
```

- 第二次调用传入 None,自动从检查点恢复

实战案例：代码审核系统

需求分析

项目目标:构建一个AI代码生成系统，工程师通过键盘实时输入审核决策

项目目标

业务流程:

业务流程

```
START → AI生成代码 → ⏸ 暂停等待审核 → 工程师输入(approve/reject) → 提交/拒绝 → END


1
```

State设计

> 完整源码: samples/chapter9/code_review/code_review_workflow.py

```
from typing import TypedDict, Optional

class CodeState(TypedDict):
    requirement: str                # 需求描述
    code: str                       # AI生成的代码
    human_decision: Optional[str]   # 工程师决策: approve/reject
    final_code: str                 # 最终提交代码
```

关键字段: human_decision 初始为 None,工程师审核后更新为 approve/reject

核心节点

> 完整源码: samples/chapter9/code_review/code_review_workflow.py 以下仅展示关键逻辑，完整实现请参考源码。

节点2:工程师审核

> interrupt_before=["review"] 在 review 节点门前暂停，human_review 在第一次调用时并未执行。 第二次 invoke(None) 恢复后，human_decision 已被 update_state 设置， 所以 human_review 只执行一次，直接处理决策结果。 代码展示放在了主流程中（见下文「第一次调用」后的代码）。

```
def human_review(state: CodeState):
    """工程师审核节点"""
    decision = state.get("human_decision")
    print(f"工程师决策: {decision}")
    return state
```

关键设计:

关键设计

- human_review 只负责处理决策结果，不负责展示代码
- 代码展示放在 第一次 invoke() 之后（此时已拿到 result["code"]）
- 工程师看到代码后输入决策，update_state 写入状态，第二次 invoke(None) 从 review 节点恢复执行

工作流构建

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

workflow = StateGraph(CodeState)

# 添加3个节点
workflow.add_node("generate", generate_code)
workflow.add_node("review", human_review)
workflow.add_node("commit", commit_code)

# 定义流程
workflow.add_edge(START, "generate")
workflow.add_edge("generate", "review")
workflow.add_edge("review", "commit")
workflow.add_edge("commit", END)

# 编译:在review节点前暂停
checkpointer = MemorySaver()
app = workflow.compile(
    interrupt_before=["review"],
    checkpointer=checkpointer
)
```

流程:

```
START → [generate] → ⏸ 暂停点 → [review] → [commit] → END


1
```

两次调用的执行机制

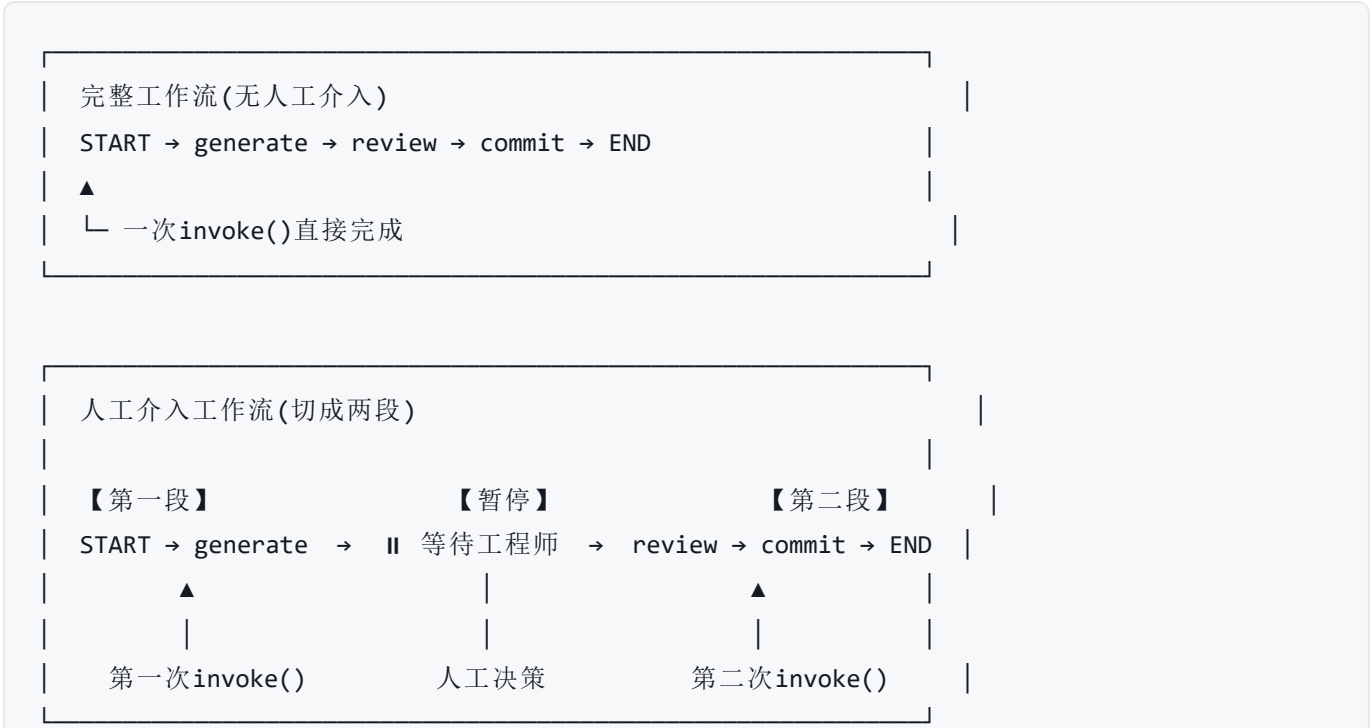
这是人工介入最核心的概念:工作流需要调用两次 invoke()。

工作流需要调用两次 invoke()

为什么需要两次调用

因为人工介入本质上是将一个完整的工作流切成两段:

将一个完整的工作流切成两段




```
config = {"configurable": {"thread_id": "session_001"}}

# 第一次调用:从START开始,执行到review节点前
result = app.invoke(
    {"requirement": "快速排序算法"},  # 传入初始输入
    config=config
)

# 此时工作流状态:
# ✓ generate节点已执行(代码已生成)
# ⏸ review节点前暂停(因为interrupt_before)
# ✗ review和commit节点尚未执行

print(result["code"])           # 可以访问已生成的代码
print(result["human_decision"]) # None(尚未决策)
```

人工决策:更新状态

```
# 工程师查看代码后做出决策
app.update_state(config, {"human_decision": "approve"})

1
2
```

update_state() 的作用:

update_state() 的作用

- 更新保存在 checkpointer 中的状态快照
- 设置 human_decision 字段，从 None 变为 approve
- 不会触发工作流继续执行,只是更新状态

不会触发工作流继续执行

```
# 第二次调用:从暂停点继续执行
result = app.invoke(
    None,      # 关键:传入None表示"从检查点恢复"
    config     # 必须使用相同的thread_id
)

# 此时工作流状态:
# ✓ 从review节点继续执行
# ✓ review节点读取到human_decision="approve"
# ✓ commit节点执行完成
# ✓ 到达END

print(result["final_code"])  # 最终提交的代码
```

- 传入 None 而不是初始输入 None 告诉LangGraph:“不要从头开始，从检查点恢复”
- 必须使用相同的 thread_id 这样才能找到第一次调用保存的状态快照

相同的 thread_id

- 这样才能找到第一次调用保存的状态快照
- 工作流从暂停点(review节点)继续执行

MemorySaver：状态快照的幕后英雄

为什么必须使用 checkpointer

在代码审核案例中，如果没有 checkpointer,工作流无法实现暂停/恢复:

```
# 错误:没有checkpointer
app = workflow.compile(interrupt_before=["review"])
# 报错:interrupt_before requires checkpointer

# ✓ 正确:提供checkpointer
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
app = workflow.compile(
    interrupt_before=["review"],
    checkpointer=checkpointer  # 必需:保存状态快照
)
```

原因:暂停/恢复需要保存和恢复工作流状态,这正是 checkpointer 的职责。而CheckPointer有很多种类型。

内存存储 vs 持久化存储:

内存存储 vs 持久化存储

```
# MemorySaver:存储在内存中(进程结束即丢失)
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver()  # 适合演示和测试

# SqliteSaver:持久化到数据库(进程结束后仍保留)
from langgraph.checkpoint.sqlite import SqliteSaver
checkpointer = SqliteSaver.from_conn_string("checkpoints.db")
```

多个暂停点的工作流

在复杂业务场景中，可能需要多个人工审核点。LangGraph 支持在多个节点前设置暂停点。

```
from langgraph.checkpoint.memory import MemorySaver

# 编译时指定多个暂停点
checkpointer = MemorySaver()
app = workflow.compile(
    interrupt_before=["review1", "review2"],  # 列表形式,多个节点
    checkpointer=checkpointer
)
```

```
START → [generate] → ⏸ 暂停点1 → [review1] → [test] → ⏸ 暂停点2 → [review2] → [deploy] → END

1
```

- 有N个暂停点就需要N+1次调用 invoke()

N个暂停点

N+1次调用

- 每次暂停后都要: update_state() 更新决策 invoke(None, config) 继续执行
- 必须始终使用相同的 thread_id

■ 学点英语

中文	English	音标	说明
人工介入	Human-in-the-Loop	/ˈhjuːmən ɪn ðə luːp/	工作流中设置暂停点等待人类审核和决策
检查点管理器	Checkpointer	/ˈtʃekpɔɪntər/	保存工作流状态快照，支持暂停后恢复
中断	Interrupt	/ˌɪntəˈrʌpt/	编译时设置 interrupt_before 在指定节点前暂停
恢复	Resume	/rɪˈzuːm/	第二次 invoke(None) 从检查点继续执行

下节预告

现在你已经掌握了 LangGraph 构建复杂工作流的能力。但你可能会发现一个新问题：每次让 AI 帮你写代码、审查代码、生成文档时，都要重复说明一遍规范和要求——"请用 Python 3.10+ 语法，遵循 PEP 8，所有函数要有类型注解。..."说了十遍、二十遍，是不是很烦？

接下来的 第10章 Skill，我们将学习如何把这些重复的指令和专业知识封装成可复用的模块，让 AI 自动遵循团队规范，就像给它配备了一套专业的"工作手册"。